主从部署与同步

配置

```
slaveof 192.168.8.203 6379
```

// 在主从切换的时候，这个配置会被重写成：

```
\# Generated by CONFIG REWRITE
replicaof 192.168.8.203 6379
```

或者在启动服务时通过参数指定 master 节点：

```
./redis-server --slaveof 192.168.8.203 6379
```

或在客户端直接执行 slaveof xx xx，使该 Redis 实例成为从节点。

启动后，查看集群状态：

```
redis> info replication
```

断开复制，就是让自己成为自己的老大：

```
redis> slaveof no one
```

此时从节点会变成自己的主节点。

主从同步分为几个阶段

一、连接阶段

1、slave node启动时（执行slaveof命令），会在自己本地保存master node的信息，包括master node的host和ip。

2、slave node内部有个定时任务replicationCron（源码replication.c），每隔 1秒钟检查是否有新的master node要连接和复制，如果发现，就跟master node建立socket网络连接，如果连接成功，从节点为该socket建立一个专门处理复制工作的文件事件处理器，负责后续的复制工作，如接收RDB文件、接收命令传播等。 当从节点变成了主节点的一个客户端之后，会给主节点发送 ping 请求。

二、数据同步阶段

3、master node第一次执行全量复制，通过bgsave命令在本地生成一份RDB 快照，将RDB快照文件发给slave node（如果超时会重连，可以调大 repl-timeout的值）。slave node首先清除自己的旧数据，然后用RDB文件加载数据。

三、命令传播阶段

问题：生成 RDB 期间，master 接收到的命令怎么处理？ 接下来就到了我们的命令传播阶段！

咕泡出品

4、master node持续将写命令，异步复制给slave node

开始生成 RDB 文件时，master会把所有新的写命令缓存在内存中（slave client buffer），在从节点加载完RDB文件后，主节点会将这个缓冲区的内容发送给从节点，

那么问题又来了！如果写入数据量很大很大的时候怎么办！

配置文件 client-output-buffer-limit

client-output-buffer-limit replica 256mb 64mb 60

这个代表命令缓冲区的上限阈值，如果命令在一定时间内超过最大我的承载，

如果60秒内缓冲区消耗持续大于64MB（持续性限制是当客户端缓冲区大小持续60秒超过64M）**

或者直接超过256MB时，主节点将直接关闭复制客户端连接

问题：如果从节点有一段时间断开了与主节点的连接是不是要重新全量复制一遍？

如果可以增量复制，怎么知道上次复制到哪里？

通过master_repl_offset记录的偏移量

redis> info replication

可以查看上次复制到哪了

master repl offset:7324556

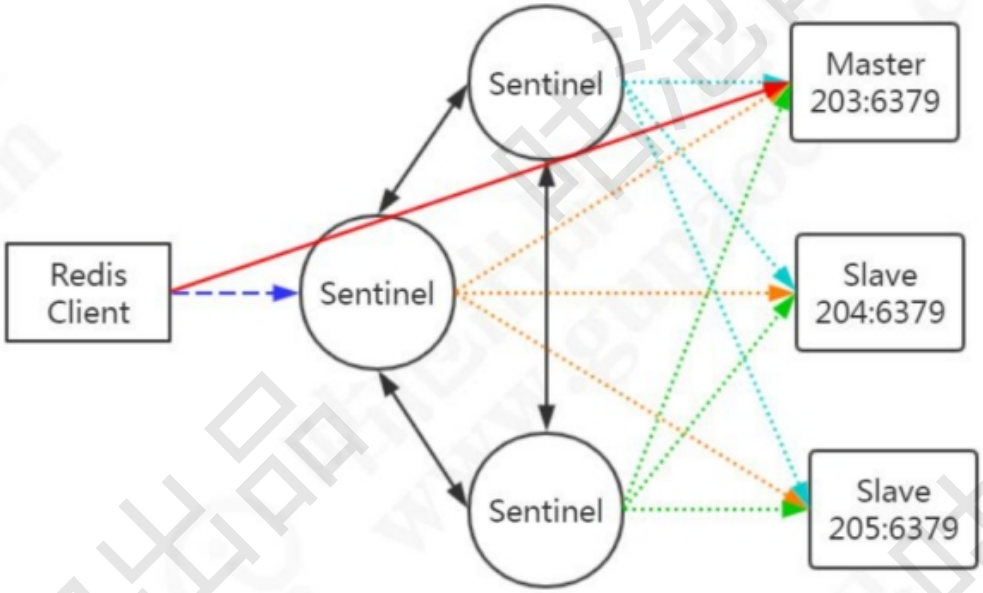
哨兵集群

现在我们知道redis怎么部署主从，它们数据是怎么做到同步的，但是，如果master挂了，我们是不是服务就不可用了，因为写不了数据了，从库只能读！

我们要每次都去手动把slave节点升级到主么，那如果半夜挂了，还得半夜跑到公司，升级的时候还会造成一段时间的服务不可用，是不是很龌。。。所以，redis为了解决这个问题，它有个sentinel哨兵机制，我相信大家都听过，但是你理解原理么，不理解没关系，我们下面就分析分析下它的哨兵！

哨兵是个什么呢。其实sentinel你就可以理解为一个监管体系，监管着你这个redis的所有服务，不管你是主也好，从也好！就像我们看的电影里面，黑社会老大的背后永远都有一个帮会社团或者委员会，凌驾在老大之上！如果你这个老大干得不行了，那么也可以把你这个老大的位子让出来！

但是，如果只有1个sentinel监管的话，如果这个哨兵挂了怎么办，那是不是也会有问题，就像帮会的社团，肯定也是有很多很多人组成的。所以sentinel它本身也是一个集群部署的，这些sentinel除了监控所有的redis服务外，它们也会互相监控！大概就是这样子的



我们来看下redis怎么实现的！我们会启动一个或者多个 Sentinel 的服务（通过 src/redis-sentinel），它本质上只是一个运行在特殊模式之下的Redis，Sentinel通过info 命令得到被监听 Redis机器的 master，slave 等信息。

那它到底是怎么能实现对redis服务的一个主动下线并把slave升级的呢，我们来看下

1. 如果Sentinel 默认以每秒钟1次的频率向Redis所有服务节点发送 PING 命令。如果在down-after-milliseconds 内都没有收到有效回复，Sentinel会将该服务器标记为下线（主观下线）。不配默认是30秒！

sentinel.conf

sentinel down-after-milliseconds

2. 这个时候Sentinel节点会继续询问其他的Sentinel节点，确认这个节点是否下线， 如果设置的quorum个 Sentinel节点都认为master下线，master才真正确认被下线（客观下线）,大家这里注意一个点

那么下线后，我们又得做事情了，做什么事情?

我们是不是得选一个slave节点来接替master节点！那么就出现2个问题了，谁去选？选谁?

这么多Sentinel，我到底让谁去选！因为Sentinel不像redis集群一样有主从的概念！它们都是平等的！

那么选谁！2个从节点，我选哪个从节点升上来！

Sentinel选举

第一步，选举sentinel大佬，那么它就用到了一个raft算法，大家有兴趣的可以私下或者课后去理解下这个raft算法，这里我就告诉大家raft的一个思想：先到先得，少数服从多数。

先到先得，就是一旦有个sentinel设置了领头sentinel，以后要求设置sentinel为领头请求都会被拒绝。跟美国总统候选人不一样，美国总统可以有多个候选人，但是这个sentinel只要有1个人说我要领头了，那么其他人就得等着！等我票拉完了你再来拉票!

少数服从多数,就是发现master挂掉的那个sentinel，会向所有其他的sentinel拉票，发送is-master-down-by-addr命令，你选我做老大，并且每人有且只有一张票，那么只要超过所有sentinel节点的半数后，它就会成为这个sentinel的老大！

防止脑裂：

因为网络问题，某个sentinel集群无法感知到 master 的存在，就会将某一个 slave 节点提升为 master 节点，此时就存在两个不同的 master节点，就像一个大脑分裂成了两个。

集群脑裂问题中，如果客户端还在基于原来的 master 节点继续写入数据，那么新的master 节点将无法同步这些数据，造成数据丢失！

后续新master跟旧的master通信后，旧的会变成从

```
# master 至少有 x 个副本连接。
  min-replicas-to-write 3   健康的slave个数小于这个数，就禁止写入
# 数据复制和同步的延迟不能超过 x 秒。
  min-replicas-max-lag 10
```

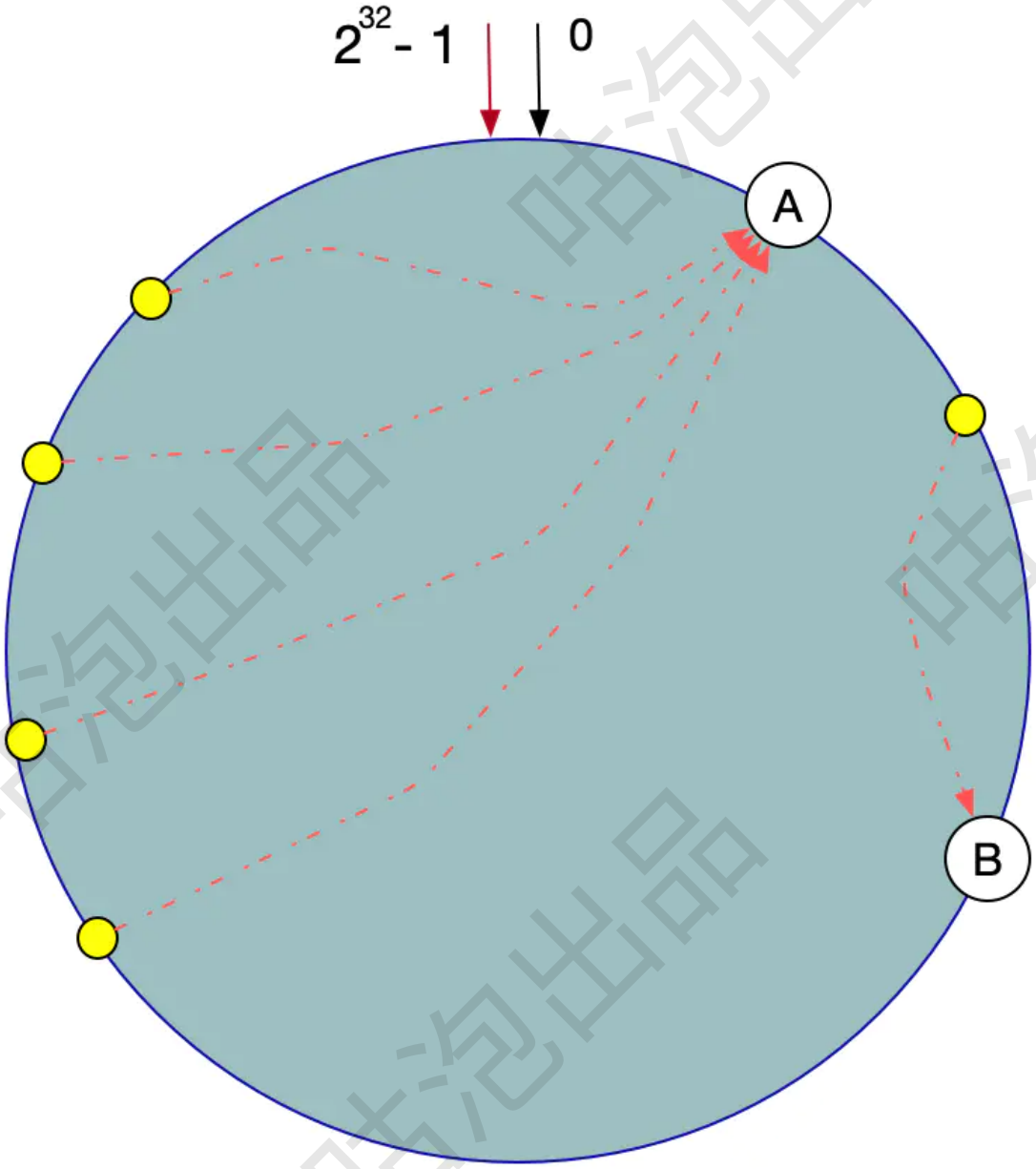
Cluster集群

数据分布

Redis既没有用哈希取模，也没有用一致性哈希，而是用虚拟槽来实现的。

hash取模： 每次更改删除节点、数据处理很麻烦。

一致性hash: 取模不是取服务器的数量，而是取模2的32方，这样添加删除节点只会影响相邻节点，但是会导致数据不均匀，节点少的时候。



所以，redis用的是虚拟槽的概念。

没那么多节点，我就变成虚拟槽 Redis创建了16384个槽（slot），每个节点负责一定区间的slot

为什么是16384?

节点我们之间都是需要心跳的。。。，以便让节点知道当前集群信息，16384=16k，在发送心跳包时使用bitmap压缩后是2k（2 * 8 (8 bit) * 1024(1k) = 2K），

然使用CRC16算法最多可以分配65535（2^16-1）个槽位，65535=65k，压缩后就是8k（8 * 8 (8 bit) * 1024(1k) = 8K）；

同时，由于其他设计折衷，Redis集群不可能扩展到超过1000个节点。

对象分布到Redis节点上时，对key用CRC16算法计算再%16384，得到一个slot的值，数据落到负责这个slot的Redis节点上。

edis的每个master节点都会维护自己负责的slot。用一个bit序列实现，比如：序列的第0位是1，就代表第一个slot是它负责；序列的第1位是0，代表第二个slot不归它负责。

redis-cli -p 7291
redis-cli -p 7292
redis-cli -p 7293

查看key属于哪个slot：

redis> cluster keyslot qingshan

注意：key与slot的关系是永远不会变的，会变的只有slot和Redis节点的关系

问题：怎么让相关的数据落到同一个节点上？

比如有些multi key操作是不能跨节点的，例如用户2673的基本信息和金融信息？

在key里面加入{hash tag}即可。Redis在计算槽编号的时候只会获取{}之间的字符串进行槽编号计算，这样由于上面两个不同的键，{}里面的字符串是相同的，因此他们可以被计算出相同的槽。

user{2673}base=

cluster主从切换

当slave发现自己的master变为FAIL状态时，便尝试进行Failover，以期成为新的master。由于挂掉的master可能会有多个slave，从而存在多个slave竞争成为master节点的过程， 其过程如下：

1.slave发现自己的master变为FAIL

2.将自己记录的集群currentEpoch加1，并广播FAILOVER_AUTH_REQUEST信息

3.其他节点收到该信息，只有master响应，判断请求者的合法性，并发送FAILOVER_AUTH_ACK，对每一个epoch只发送一次ack

4.尝试failover的slave收集FAILOVER_AUTH_ACK

5.超过半数后变成新Master

6.广播Pong通知其他集群节点。

Redis Cluster既能够实现主从的角色分配，又能够实现主从切换，相当于集成了Replication和Sentinel的功能。

redission分布式锁

其实我们用redis做分布式锁有问题的！！什么问题？我们假如说redis是主从，然后我主去setnx一个值，当我没同步到从的时候，redis主就挂了，从就升为主，但是这个新来的主是没有这个锁的，也能添加成功！！

这个问题怎么解决？解决不了，因为redis本来就是ap的，cap理论，ap，保证可用性，但是不保证强数据一致性！！！

要么用ZK,zk是cp，zk的主从切换，必须要等到数据同步完成！！

但是虽然单主解决不了，所以redis官网给了个建议，用集群！！

然后集群用一个红锁

红锁实战

```
RLock rLock=redissonClient.getLock("updateAccount");
RLock rLock1=redissonClient.getLock("updateAccount");
RedissonRedLock redLock=new RedissonRedLock(rLock,rLock1);
// 最多等待100秒、上锁10s以后解锁
if(redLock.tryLock(1,10, TimeUnit.SECONDS)){
    System.out.println("获取锁成功");
}else
{
    System.out.println("获取锁失败");
}
Thread.sleep(20000);
rLock.unlock();

redissonClient.shutdown();
```

我们发现，红锁可以传多个实例！！

解锁

```
protected RFuture<Boolean> unlockInnerAsync(long threadId) {
    // 释放锁时需要在redis实例上执行的lua命令
    return commandExecutor.evalWriteAsync(getName(), LongCodec.INSTANCE,
RedisCommands.EVAL_BOOLEAN,
            // 如果分布式锁KEY不存在，那么向channel发布一条消息
            "if (redis.call('exists', KEYS[1]) == 0) then " +
                "redis.call('publish', KEYS[2], ARGV[1]); " +
                "return 1; " +
            "end;" +
            // 如果分布式锁存在，但是value不匹配，表示锁已经被占用，那么直接返回
            "if (redis.call('hexists', KEYS[1], ARGV[3]) == 0) then " +
                "return nil;" +
            "end; " +
            // 如果就是当前线程占有分布式锁，那么将重入次数减1
            "local counter = redis.call('hincrby', KEYS[1], ARGV[3], -1); " +
            // 重入次数减1后的值如果大于0，表示分布式锁有重入过，那么只设置失效时间，还不能删
除
            "if (counter > 0) then " +
                "redis.call('pexpire', KEYS[1], ARGV[2]); " +
                "return 0; " +
            "else " +
```

```
                // 重入次数减1后的值如果为0，表示分布式锁只获取过1次，那么删除这个KEY，并发布解锁消息
                "redis.call('del', KEYS[1]); " +
                "redis.call('publish', KEYS[2], ARGV[1]); " +
                "return 1; "+
            "end; " +
            "return nil;",
            // 这5个参数分别对应KEYS[1]，KEYS[2]，ARGV[1]，ARGV[2]和ARGV[3]
            Arrays.<Object>asList(getName(), getChannelName(),
LockPubSub.unlockMessage, internalLockLeaseTime, getLockName(threadId));

}
```

结论：如果不能容忍一定问题的，那么就用cp的zk，如果能容忍，就用redis!!

分段锁

出现分段锁的背景：

1个请求过来去尝试拿锁，这样我们的并行会变成串行，再一定程度上影响性能，所以能不能想一些办法去解决一下！！

那么就有了分段锁的概念！！

分段锁分段锁，我们最先想到的是jdk1.7中的chm里面的分段锁的概念，就是我锁一个区间

其实就是把数据分段，然后再段上加锁，但是在段上加锁也不能有业务问题，如果有，就不能用分段锁

比如：我们一般做的库存扣减

我们假如有100个库存，我们如果没有分段的话，是一个一个的去拿锁扣库存！

那么我是不是可以把100个库存分5个地方存，每个地方20个，那我就能支持同时5个线程来扣减库存！

面试题

有没有用过Redis,你一般在哪里用到?

缓存、跟memcache的区别，主要区别redis属于分布式，memcache单机

分布式锁、分布式ID等等

单点登录session缓存

Redis有哪些数据类型

常用的有String、hash、list、set、zset 还有不经常用的比如基数统计HyperLogLog、经纬度geo,stream

String 分布式ID，分布式锁等等

Hash 可以存储结构型的对象，单独操作某个字段更加方法

List 元素可重复的有序列表，可以充当队列、消息列表等

set 无序不重复的集合，可以做交集、并集、差集等等

redisKey的最大容量是多少，value存储最大容量是多少?

key 512M

value 根据数据类型来，string 是512M，list、hash、2的32次方-1（40亿）

Redis为什么快?

1.纯内存操作

2.单线程操作

3.数据结构简单，底层hash，跳表、压缩列表、快速列表等

4.非阻塞IO多路复用机制

为什么Redis用单线程就快

因为瓶颈不在CPU，在于机器的内存跟带宽，单线程避免了上下文切换,更快

既然Redis是内存型数据库，那么Redis的数据如果满了怎么办?

过期 被动过期跟定期过期

被动过期，在数据访问的时候会判断是否已经过期，如果过期，移除

定期过期，会在serverCorn()中去检测里面是否有过期的key，

1. 从设置了过期时间的数据中最多每次取20个桶。。。。桶里面可能有多条数据

（ACTIVE_EXPIRE_CYCLE_KEYS_PER_LOOP配置的

值为20，active-expire-effort可以更改大小）

2. 删除这20个桶中过期的键
3. 如果没有抽样到数据，或者过期数据超过抽样比例数据的10%，重复步骤 1 和 2,每16次执行时间检查，如果超时，那么跳出！

淘汰 8大淘汰算法，以及说明，默认是no-enviction

1. volatile-lru：从已设置过期时间的数据集（server.db[i].expires）中挑选最近最少使用的数据淘汰
2. volatile-ttl：从已设置过期时间的数据集（server.db[i].expires）中挑选将要过期的数据淘汰

3. volatile-random：从已设置过期时间的数据集（server.db[i].expires）中任意选择数据淘汰
4. allkeys-lru：从数据集（server.db[i].dict）中挑选最近最少使用的数据淘汰
5. allkeys-random：从数据集（server.db[i].dict）中任意选择数据淘汰
6. no-enviction（驱逐）：禁止驱逐数据，新写入操作会报错

我们知道LFU算法是最少使用，根据次数来，那么我假如有个key在一年前使用次数很高，那是怎么去淘汰的？可以配置多少分钟没访问就减少多少次

redis是内存型数据，那怎么解决持久化的？

RDB 快照

触发机制

save 900 1 #900秒内如果超过1个key被修改，则发起快照保存。
save 300 10 #300秒内如果超过10个key被修改，则快照保存

save 、bgSave手动触发

AOF 追加

他们的区别是什么？

aof比rdb更新频率高，优先使用aof还远数据

aof比rdb更安全也更大

rdb性能比aof好

如果2个都配置了优先用aof

什么是缓存雪崩、缓存穿透、缓存击穿？你怎么解决？

缓存雪崩 缓存雪崩就是Redis的大量热点数据同时过期（失效），因为设置了相同的过期时间，刚好这个时候Redis请求的并发量又很大，就会导致所有的请求落到数据库。

1）加互斥锁或者使用队列，针对同一个key只允许一个线程到数据库查询

2）缓存定时预先更新，避免同时失效
3）通过加随机数，使key在不同的时间过期

缓存穿透 缓存穿透是指缓存和数据库中都没有的数据，但是用户一直请求不存在的数据！这时的用户很可能就是攻击者，恶意搞你们公司的，攻击会导致数据库压力过大。

布隆过滤器，位图里面只有0跟1，然后将访问的值通过多个不同的hash算法得到不同的值，放置在不同的位图标记为1

有误判，没法判断数据一定存在，但是能判断一定不存在！

封IP

缓存击穿

单个key过期的时候有大量并发，使用互斥锁，回写redis

一般公司不会考虑

什么是缓存数据一致性问题?怎么解决的?

什么是数据一致性，在并发环境下，可能会导致redis跟DB的数据产生不一致

如果是更新缓存

1.线程A 把DB值从1更新成2，去更新缓存（还没执行）

2.线程B把DB值从2更新成3

3.线程B缓存更新成了3

4.线程A又把值变成了2

我们看，DB是3.而redis是2，导致了数据不一致

先删除缓存行不行

1.线程A删除缓存，再去更新DB（1-2）

2.线程B来查询缓存，发现缓存不存在，我从DB获取值更新到redis，redis变成1

3.线程A把DB值从1更新成2

后删除缓存行么

1.线程A请求缓存，没有缓存，从DB拿到1,还没更新到缓存

2.线程B将1更新为2，并且删除缓存，DB的值为2

3.线程A更新缓存，redis为1

所以，不管怎么样都会有缓存一致性问题，那么怎么解决呢，就是延时双删！我在更新DB之前先删除缓存，更新DB之后再删除缓存！

为了解决后删除缓存的场景，所以，需要延时！

Redis的集群方案有哪些？用的是哪个？优势在哪里

1.twemproxy，大概概念是，它类似于一个代理方式， 使用时在本需要连接 redis 的地方改为连接 twemproxy， 它会以一个代理的身份接收请求并使用一致性 hash 算法，将请求转接到具体 redis，将结果再返回 twemproxy。
缺点： twemproxy 自身单端口实例的压力，使用一致性 hash 后，对 redis 节点数量改变时候的计算值的改变，数据无法自动移动到新的节点。

2.codis，目前用的最多的集群方案，基本和 twemproxy 一致的效果，但它支持在 节点数量改变情况下，旧节点数据可恢复到新 hash 节点

3.redis cluster 3.0自带的集群，特点在于他的分布式算法不是一致性 hash，而是 hash 槽的概念,16384个slots，以及自身支持节点设置从节点。具体看官方文档介绍

优点：无中心架构

原生　不需要其他的代理或者工具

易扩展　添加或者移除新节点，不需要停机，易扩展

高可用 实现了一个类似 Raft 的共识方式，来保障整个集群的可用性

缺点： 需要客户端的支持、只支持一个数据库、批量多个key操作受限，因为可能不在一个slot

4.哨兵

优点：

Redis Sentinel 集群部署简单;

能够解决 Redis 主从模式下的高可用切换问题;

很方便实现 Redis 数据节点的线形扩展，轻松突破 Redis 自身单线程瓶颈，可极大满足 Redis 大容量或高性能的业务需求;

可以实现一套 Sentinel 监控一组 Redis 数据节点或多组数据节点。

缺点：

部署相对 Redis 主从模式要复杂一些，原理理解更繁琐;

资源浪费，Redis 数据节点中 slave 节点作为备份节点不提供服务;

Redis Sentinel 主要是针对 Redis 数据节点中的主节点的高可用切换，对 Redis 的数据节点做失败判定分为主观下线和客观下线两种，对于 Redis 的从节点有对节点做主观下线操作，并不执行故障转移。

不能解决读写分离问题，实现起来相对复杂。

redis中String的底层数据结构是什么？优势在哪里

SDS

因为C语言本身没有字符串类型，只能用字符数组char[]实现。

1、使用字符数组必须先给目标变量分配足够的空间，否则可能会溢出。

2、如果要获取字符长度，必须遍历字符数组，时间复杂度是O(n)。

3、C字符串长度的变更会对字符数组做内存重分配。

4、通过从字符串开始到结尾碰到的第一个'\0'来标记字符串的结束，因此不能保存图片、音频、视频、压缩文件等二进制(bytes)保存的内容，二进制不安全。

SDS的特点：

1、不用担心内存溢出问题，如果需要会对SDS进行扩容。

2、获取字符串长度时间复杂度为O(1)，因为定义了len属性。

3、通过“空间预分配”（ sdsMakeRoomFor）和“惰性空间释放”，防止多次重分配内存。

4、判断是否结束的标志是len属性，可以包含'\0'（它同样以'\0'结尾是因为这样就可以使用C语言中函数库操作字符串的函数了）。

说说Redis的哈希槽概念

官方cluster方案中没有使用一致性 hash，而是引入了哈希槽的概念，Redis 集群有 16384 个哈希槽，每个 key 通过 CRC16 校验后对 16384 取模来决定放置哪个槽，集群的每个节点负责一部分 hash 槽。

对key用CRC16算法计算再%16384，得到一个slot的值

redis的分布式锁怎么实现

1.setnx+lua脚本,基于redis的单线程命令执行

2.redission封装的分布式锁，底层也属于lua脚本

redis主从复制过程以及原理

1、slave node启动时（执行slaveof命令），会在自己本地保存master node的信息，包括master node的host和ip。
2、slave node内部有个定时任务replicationCron（源码replication.c），每隔 1秒钟检查是否有新的master node要连接和复制，如果发现，就跟master node建立socket网络连接，如果连接成功，从节点为该socket建立一个专门处理复制工作的文件事件处理器，负责后续的复制工作，如接收RDB文件、接收命令传播等。 当从节点变成了主节点的一个客户端之后，会给主节点发送 ping 请求。
二、数据同步阶段
3、master node第一次执行全量复制，通过bgsave命令在本地生成一份RDB 快照，将RDB快照文件发给slave node（如果超时会重连，可以调大 repl-timeout的值）。slave node首先清除自己的旧数据，然后用RDB文件加载数据。
三、命令传播阶段

4、master node持续将写命令，异步复制给slave node
开始生成 RDB 文件时，master会把所有新的写命令缓存在内存中（slave client buffer），在从节点加载完RDB文件后，主节点会将这个缓冲区的内容发送给从节点，

Redis哨兵机制如何实现故障自动转移

检查下线

1.如果Sentinel 默认以每秒钟1次的频率向Redis所有服务节点发送 PING 命令。如果在down-after-milliseconds 内都没有收到有效回复，Sentinel会将该服务器标记为下线（主观下线）。不配默认是30秒！

sentinel.conf

sentinel down-after-milliseconds

2.这个时候Sentinel节点会继续询问其他的Sentinel节点，确认这个节点是否下线， 如果设置的quorum个 Sentinel节点都认为master下线，master才真正确认被下线（客观下线）

我们看下，我们sentinel只配置了master地址，他们到底是怎么相互通信的

1.主要通过sentinel的3个命令实现跟实例通信以及各哨兵之间的通信

2.ping命令，ping 配置的实例地址

3.info命令，获取到配置实例地址的状态与信息！能获取到主从实例相关信息，以及实例的健康信息

4.publish命令，各sentinel之间的通信通过订阅模式，哨兵向实例的HELLO频道发布有关自己以及主

节点的信息，也就是所谓的HELLO消息。所有哨兵都会订阅主节点和从节点的HELLO频道，因此，每个哨兵都会收到其他哨兵发布的信息。

选举

第一步，选举sentinel大佬，那么它就用到了一个raft算法，大家有兴趣的可以私下或者课后去理解下这个raft算法，这里我就告诉大家raft的一个思想：先到先得，少数服从多数。

少数服从多数,就是发现master挂掉的那个sentinel，会向所有其他的sentinel拉票，发送is-master-down-by-addr命令，你选我做老大，并且每人有且只有一张票，那么只要超过所有sentinel节点的半数后，它就会成为这个sentinel的老大！

半数半数，所以我们的sentinel节点一般是部署奇数台，不然。。。就会脑裂，脑裂就是假如每个竞选者都是50%票，它就不知道要干嘛了！

先到先得，竞选后，谁先得到超过半数，那么就会成为leader,其他的sentinel就不能成为leader

变成leader后选哪个从变master

分别是断开连接时长、优先级排序、复制数量、进程 id。

如果与哨兵连接断开的比较久，超过了某个阈值，就直接失去了选举权。如果拥有选举权，那就看谁的优先级高，这个在配置文件里可以设置（replica-priority 100），数值越小优先级越高。

如果优先级相同，就看谁从master中复制的数据最多（复制偏移量最大），选最多的那个，这个偏移量就是我们刚才讲的主从复制的时候复制到哪里的那个东西！

如果复制数量也相同，就选择进程id最小的那个。

所以我们再来总结下Sentinel的作用有哪些

1. 监控：Sentinel 会不断检查主服务器和从服务器是否正常运行
2. 故障处理：如果主服务器发生故障，Sentinel可以启动故障转移过程。把某台服务器升级为主服务器，并发出通知
3. 配置管理：客户端连接到 Sentinel，获取当前的 Redis 主服务器的地址。我们不是直接去获取redis主服务的地址，而是根据sentinel去自动获取谁是主机，不然发生故障后我们还得改代码的连接，很麻烦！

说下你对Redis rehash的理解

有2个hashTable，采用链地址法解决hash冲突

ht[0].used/ht[0].size>=1

|| ht[0].used/ht[0].size>5

原有hash的2倍

1、ht[1]分配空间，空间的大小由伸缩还是缩小以及原本的ht[0].used值决定
如果扩展分配空间为第一个大于等于2*ht[0].used的2^n的数
如果收缩分配空间为第一个大于等于ht[0].used的2^n的数
负载因子 load_factor = ht[0].used/ht[0].size

2、重新计算ht[0]上所有节点的hash值，取模后放到ht[1]中。

3、释放ht[0]的空间，交换ht[0]、ht[1]的指针

程序什么时候进行扩展和收缩呢？
当没有BGSAVE/BGREWRITEAOF命令执行时，负载因子大于1，若有则负载因子大于5.
因为操作时会fork()子进程，服务器不想要频繁rehash，所以提高阈值。

咕泡出品